

CYBER SECURITY CTF MACHINE DEVELOPMENT REPORT

Comprehensive Build Documentation, Hardening Review, and Vulnerability Exploit Walkthrough

Prepared By: Mohammed Furquan Ullah
Department: Department of Computer Applications
Date: 29-June-2026
Target System Static IP: 192.168.8.136
Configured Hostname: furqan-server

1. Objective

The primary purpose of this project is to construct, configure, secure, and document an operational Capture The Flag (CTF) Linux laboratory instance. Designed from a defensive administration framework, the target sandbox architecture mimics real-world insecure deployments—specifically focusing on administrative oversights such as technical information leak gaps, weak authentication mechanics, and local permission escalation paths. This project fulfills academic development metrics covering network interface isolation, firewall management policies, and interactive challenge verification chains.

2. Machine Specifications

To support high performance during testing phases, the target virtual environment was built using structured virtualization limits:

- **Hypervisor Environment Platform:** VMware Workstation Pro / vSphere Node Infrastructure
- **Virtual Processor Provisioning:** 2 vCPU Cores Architecture
- **Volatile Memory Base (RAM):** 4.0 GB Allocation Memory
- **Virtual Storage Target Size:** 25 GB Dynamically Allocated Storage Capacity

3. Operating System Used

The environment runs an unhardened server installation of **Ubuntu 26.04 LTS (64-bit architecture)** operating on stable Linux Kernel distribution baseline 7.0.0-27-generic x86_64. Ubuntu was selected as the underlying system because it allows for direct control over security boundaries, user profiling structures, service rulesets, and log monitors.

4. Network Configuration

To safely isolate exploit payloads within local boundaries, the target was assigned a single virtual Host-Only host adapter interface. The configuration details include:

- **Active Adapter Card Interface:** `ens33`
- **Assigned Laboratory Static IP:** 192.168.8.136
- **Internal Subnet Broadcast Mask:** 255.255.255.0
- **DHCP IP Bounds Range:** Statistically mapped inside local testing networks to prevent IP drift.

5. User Accounts Created

To fulfill the criteria requiring three distinct user instances, the system has three fully provisioned interactive user environments with localized home paths:

Username	Role Identity Type	Directory Environment Profile Path	Intended Security Purpose
tim	Low-Privilege Account	<code>/home/tim/</code>	Acts as the initial remote compromise vector via web-discovered credentials.
jhon	Mid-Tier Isolated Profile	<code>/home/jhon/</code>	Maintains an unprivileged local environment to test lateral asset reconnaissance.
alex	High-Privilege Admin User	<code>/home/alex/</code>	Maintains full vertical escalation paths and local administrative commands.

6. Services Configured

To optimize performance and minimize background noise, only two active services were exposed on the external interfaces:

- **Secure Shell Service Daemon (SSH):** Running OpenSSH Server Engine version 10.2p1 on port 22 to handle initial terminal footholds.
- **HTTP Web Daemon:** Driven via the Apache HTTPD Server process version 2.4.66 on port 80 to expose the front-end profile.

7. Website Design Overview

The front-end interface hosted under `/var/www/html/` features a professional developer profile portfolio. It highlights academic background steps, standard development skills, active industry certifications, and completed repository projects. The content mimics a production developer platform to cleanly blend with the challenge elements hidden within its code directories.

8. Hidden Credentials Methodology

To reward thorough inspection, player access tokens are split into separate stages instead of being exposed directly on the root landing page:

- **The Disallowed Endpoint:** The system maps an active `robots.txt` parameter pointing to an isolated directory structure: `/secret_development_login/`.
- **The Debug Leak String:** Accessing the folder exposes a developer note detailing that a profile clone step was achieved. This text leaks the unencrypted username and password pair: `tim / webmaster_secure_pass_2026`.

9. Steganography Implementation

An image file named `challenge.jpg` (6,807 bytes) was placed on the low-privilege desktop path at `/home/tim/Desktop/photo.jpg`. This asset functions as a challenge checkpoint. Using `steghide`, a professional candidate resume PDF document was embedded inside the pixel matrix. To extract it, analysts use the password hint found during web exploration, testing their file carving and metadata analysis skills.

10. Flag Locations

- **User Milestone Flag Target:** Mapped at path `/home/tim/user.txt`
Token Value String: `CTF{webmaster_access_granted_7712}`
- **Administrative Privilege Root Flag:** Mapped at path `/root/root.txt`
Token Value String: `FLAG{su hail_root_access_granted_system_pwned_5541}`

11. Testing Procedure

Sanity tests were executed by running black-box scans against the host from a remote testing machine. First, port discovery tools confirmed that attempts to interact with unapproved listening paths were dropped by the firewall. Next, access test scripts confirmed that the `webmaster` shell interface restricts standard reads on `/root/root.txt`, verifying the privilege boundaries are functioning correctly.

12. Vulnerability Walkthrough

The complete exploitation path proceeds through four distinct stages:

- Information Gathering:** A network scan reveals ports 22 (SSH) and 80 (HTTP) are open.
- Web Enumeration:** Inspecting `/robots.txt` reveals the disallowed directory `/secret_development_login/`, which contains the unencrypted credentials for the `webmaster` user.
- Foothold:** Authenticating via SSH using these credentials provides initial shell access and uncovers the user milestone flag.
- Privilege Escalation:** Reviewing local configurations reveals an option to pivot to the secondary account `furqan`. Authenticating with the password `furqan123` grants full administrative root privileges, allowing the tester to capture the final flag file.

13. Risk Analysis and Mitigation Matrix

Identified System Flaw	Risk Rating	Technical Mitigation Action Strategy
Cleartext Authentication Keys in Web Assets	CRITICAL	Remove plain text credentials from source configurations; implement centralized secrets management vaults.
Sensitive Endpoint Disclosure via Public Directives	HIGH	Remove administrative paths from <code>robots.txt</code> ; enforce strict access control policies on development endpoints.
Password-Based Remote Authentication Weakness	MEDIUM	Disable interactive password logins on SSH profiles; enforce cryptographic key-pair authentication.

14. Conclusion

The deployed laboratory environment successfully meets all project guidelines. By moving plain credentials into isolated directory paths, the platform requires students to perform thorough network mapping, source parsing, hidden asset extraction, and privilege exploration to fully compromise the target system.